

Inspiracja – symulowany samochód autonomiczny

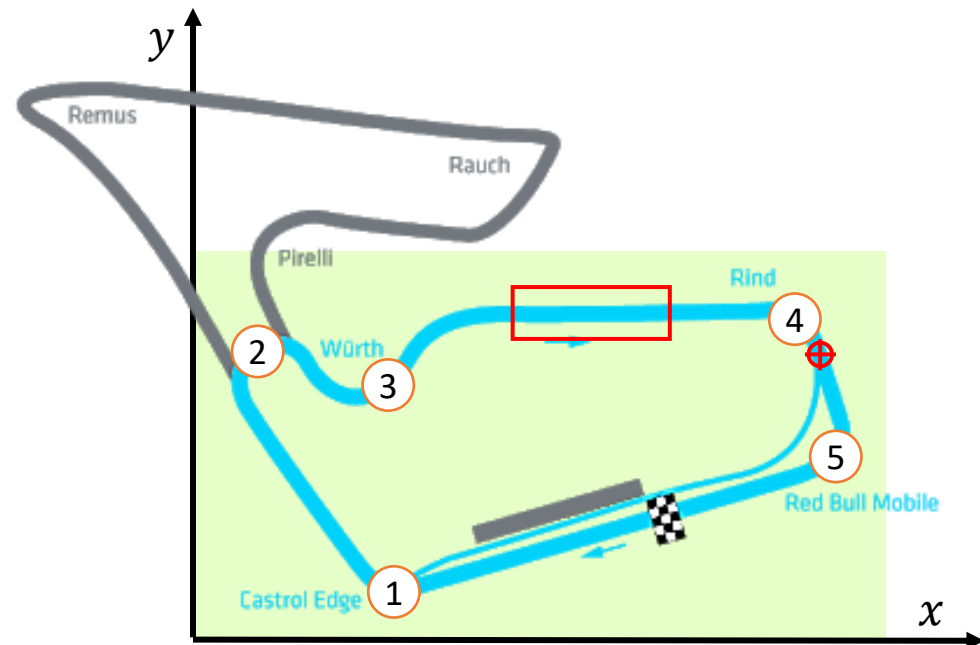
pythonprogramming.net



Plansze, scenariusz

- PNG 1920x1080, kanały RGB
- skala 22 piksele = 1 m
- szerokość toru i margines od innych elementów 3-4 m
- pętla jednokierunkowa, możliwe skrzyżowania
- plansza dodatkowa z takimi samymi zakrętami

- scenariusz = ciąg etapów realizowanych kolejno
- w etapie – trening pokonywania fragmentu topologii
- etap = obszar startowy + cel
- obszar startowy musi obejmować fragment toru, może obejmować trawnik
- cel zawsze „z przodu”
- CSV, 1 scenariusz, maks. 8 etapów



```
1;1;41.77;45.55;18.14;32.05;47.32;37.5
1;1;51.00;55.95;31.91;38.00;55.64;27.7
1;2;36.64;50.55;22.64;26.36;32.73;21.3
```

①

②

③

④

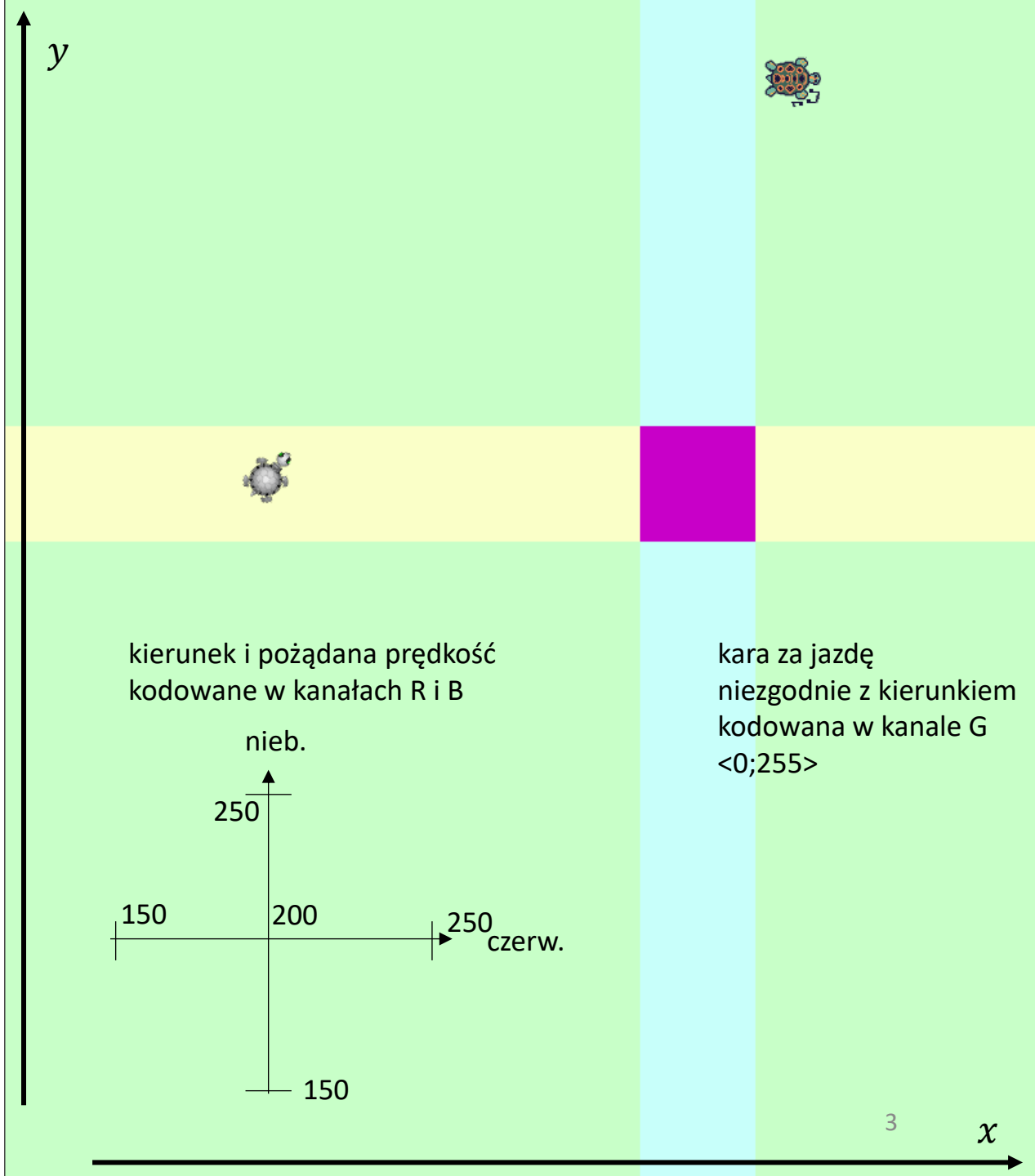
⑤

liczba agentów
uruchamianych w tym
obszarze – aktualnie 1

$id_scenariusza; 1; x_{min}^{(1)}; x_{max}^{(1)}; y_{min}^{(1)}; y_{max}^{(1)}; x_g^{(1)}; y_g^{(1)}$

Kodowanie kolorów

- Trawnik ma kolor $R=200, G=255, B=200$
- Przynajmniej 1 zakręt wyoblony
- Gradacja kolorów na łukach niewymagana
- Kanał G koduje istotność zalecanego kierunku i prędkości jazdy



Organizacja pracy

gitlab-stud.elka.pw.edu.pl



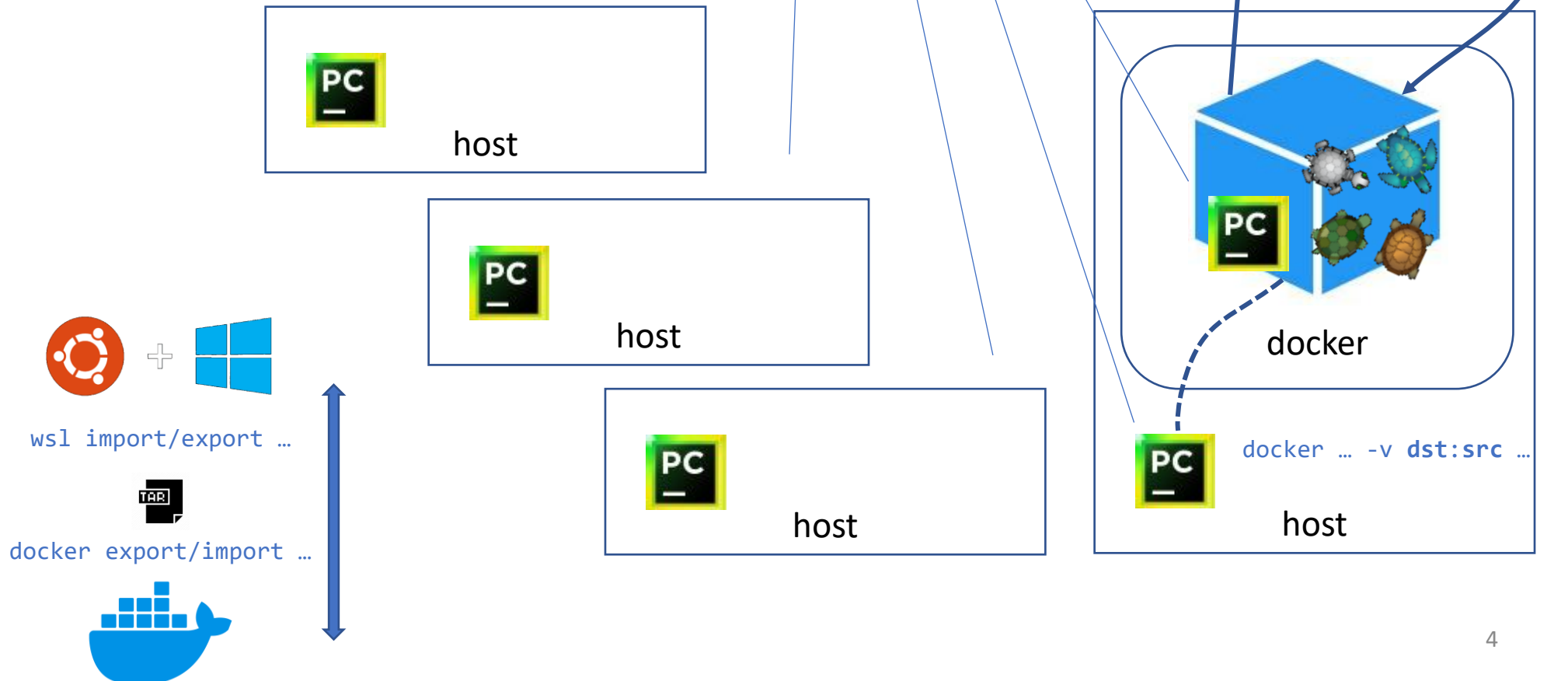
Dydaktyczny Serwer Git

dudekw/siu-20.04:latest

Docker Hub



mojprofil/siu-zespolX:etapY



Organizacja pracy – suplement – praca bez Dockera

Alternatywą dla Dockera jest Ubuntu na maszynie wirtualnej albo bezpośrednio na sprzęcie.

Aby zainstalować symulator, należy wykonać poniższe kroki (Ubuntu 20.04):

1. zainstalować ROS Noetic według instrukcji <http://wiki.ros.org/noetic/Installation/Ubuntu>
2. `mkdir -p $HOME/siu_ws/src`
3. `cd $HOME/siu_ws/src`
4. `sudo apt update`
5. `sudo apt upgrade`
6. `sudo apt install ros-noetic-teleop-twist-keyboard ros-noetic-moveit-simple-controller-manager ros-noetic-imu-sensor-controller ros-noetic-force-torque-sensor-controller ros-noetic-twist-mux ros-noetic-four-wheel-steering-controller ros-noetic-image-proc ros-noetic-range-sensor-layer ros-noetic-pointcloud-to-laserscan ros-noetic-laser-filters ros-noetic-teb-local-planner ros-noetic-gmapping ros-noetic-octomap-server ros-noetic-octomap-mapping ros-noetic-people-msgs ros-noetic-rosbridge-server ros-noetic-rospy-message-converter ros-noetic-sound-play ros-noetic-object-recognition-msgs ros-noetic-moveit-ros-planning-interface ros-noetic-move-base-msgs bison flex ros-noetic-navigation ros-noetic-moveit-planners-ompl python-scipy python-catkin-tools ros-noetic-turtlebot3* ros-noetic-ddynamic-reconfigure python3-catkin-pkg-modules python3-rospkg-modules python3-opencv python3-matplotlib python-rosinstall git`
7. `git clone -b noetic-devel https://github.com/RCPRG-ROS-pkg/ros_tutorials`
8. `cd $HOME/siu_ws`
9. `source /opt/ros/noetic/setup.bash`
10. `catkin build`

Przy tej instalacji plansza wykorzystywana przez żółwie znajduje się pod ścieżką:

`$HOME/siu_ws/src/ros_tutorials/turtlesim/images/roads.png`

Uwaga: w przypadku pracy z *Dockerem*, po zainstalowaniu tensorflow może zaistnieć potrzeba aktualizacji bibliotek obsługujących połączenie przez VNC, poleceniem:
`pip install --upgrade Jinja2 flask`

Organizacja kodu

- Kontrakty dla środowiska symulacyjnego i uczącego ustanowione w klasach bazowych
 - TurtleSimEnvBase
 - DqnBase
- Przykładowa implementacja środowiska 1-agentowego
 - TurtleSimEnvSingle
- Ucząc, sprawdzić wpływ wybranych parametrów na rozwiązanie
 - * - nie zmieniać
 - > - nie zwiększać
 - < - nie zmniejszać
- Uzupełnić brakujące fragmenty kodu:

TODO STUDENCI

...

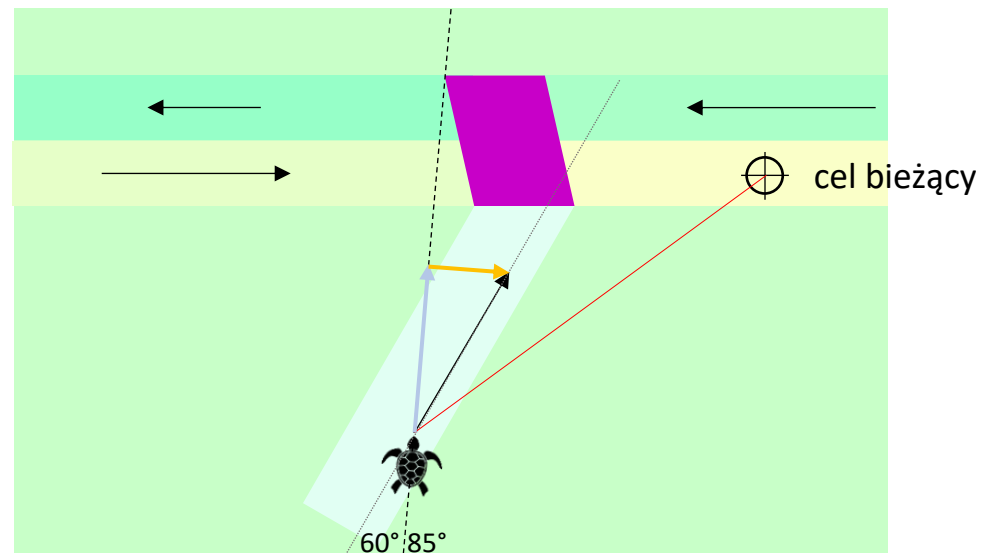
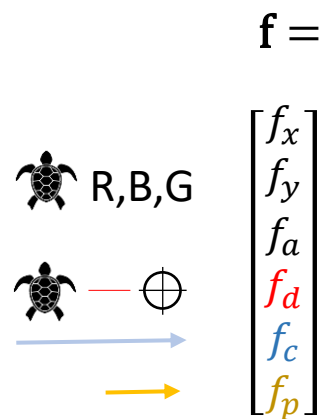
```
class TurtleAgent:
    pass

class TurtleSimEnvBase(metaclass=abc.ABCMeta):
    # określa parametry symulacji (poniższy zestaw i wartości)
    def __init__(self):
        # parametry czujnika wizyjnego i interakcji z symulacją
        self.GRID_RES = 5
        self.CAM_RES = 200
        self.SEC_PER_STEP = 1.0
        self.WAIT_AFTER_MOVE = .01
        # parametry oceny sytuacyjnej
        self.SPEED_RWRD_RATE = 0.5
        self.SPEED_RVRS_RATE = -10.0
        self.SPEED_FINE_RATE = -10.0
        self.DIST_RWRD_RATE = 2.0
        self.OUT_OF_TRACK_FINE = -10
        self.COLLISION_DIST = 1.5
        self.DETECT_COLLISION = False
        self.MAX_STEPS = 20
        self.PI_BY = 6
        uzupełnić reset()
```

```
class TurtleSimEnvSingle(TurtleSimEnvBase):
    def __init__(self):
        super().__init__()
    def step(self, actions, realtime=False):...
    uzupełnić step()
```

Świadomość sytuacyjna (klasa bazowa)

get_road(tname)



get_map(tname)

